

Il Modello Fisico

Esercizi sul Modello Fisico in MySQL



Prof. Francesco Gobbi

I.I.S.S. Galileo Galilei — Ostiglia (MN)

Esempio di modello fisico: creazione tabelle

Creiamo un piccolo database per gestire **studenti**, **corsi** ed **esami**.

DDL: creazione delle tabelle

```
CREATE TABLE studente (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    nome VARCHAR(50) NOT NULL,  
    cognome VARCHAR(50) NOT NULL,  
    classe VARCHAR(10) NOT NULL  
);
```

```
CREATE TABLE corso (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    nome VARCHAR(50) NOT NULL,  
    ore INT NOT NULL CHECK (ore > 0)  
);
```

Esempio di modello fisico: creazione tabelle

Creiamo un piccolo database per gestire **studenti**, **corsi** ed **esami**.

DDL: creazione delle tabelle

```
CREATE TABLE studente (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    nome VARCHAR(50) NOT NULL,  
    cognome VARCHAR(50) NOT NULL,  
    classe VARCHAR(10) NOT NULL  
);
```

```
CREATE TABLE corso (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    nome VARCHAR(50) NOT NULL,  
    ore INT NOT NULL CHECK (ore > 0)  
);
```

Esempio di modello fisico: inserimento dati

Dopo aver creato la struttura del database, inseriamo alcuni dati nelle tabelle.

DML: inserimento nelle tabelle studente e corso

```
INSERT INTO studente (nome, cognome, classe) VALUES
('Mario', 'Rossi', '5IA'),
('Luca', 'Bianchi', '5IA'),
('Anna', 'Verdi', '5IB'),
('Giulia', 'Neri', '5IB'),
('Paolo', 'Gialli', '5IC');
```

```
INSERT INTO corso (nome, ore) VALUES
('Database', 40),
('Programmazione Web', 35),
('Sistemi e Reti', 45),
('Informatica Teorica', 30);
```

Esempio di modello fisico: inserimento esami

DML: inserimento nella tabella esame

```
INSERT INTO esame (voto, dataEsame, studenteId, corsoId) VALUES
  (8, '2025-04-10', 1, 1),
  (7, '2025-04-12', 2, 1),
  (9, '2025-04-15', 1, 2),
  (6, '2025-04-18', 3, 3),
  (10, '2025-04-20', 4, 2),
  (8, '2025-04-22', 2, 3),
  (5, '2025-04-23', 5, 1),
  (9, '2025-04-24', 3, 2),
  (7, '2025-04-25', 4, 4);
```

- ▶ studenteId e corsoId permettono di collegare le tre tabelle.
- ▶ Negli esercizi useremo l'**equi join** con condizioni nella clausola WHERE.

Esercizio 1

Testo

Visualizzare nome, cognome e classe di tutti gli studenti.

Idea di soluzione

- ▶ I dati richiesti si trovano tutti nella tabella studente.
- ▶ Non serve ancora collegare più tabelle.
- ▶ Bisogna usare una query `SELECT`.

Soluzione esercizio 1

Query

```
SELECT nome, cognome, classe  
FROM studente;
```

Spiegazione

- ▶ SELECT indica quali colonne visualizzare.
- ▶ FROM studente indica la tabella da cui leggere i dati.
- ▶ Non c'è WHERE, quindi vengono mostrate tutte le righe.

Testo

Visualizzare nome e cognome degli studenti della classe 5IA.

Idea di soluzione

- ▶ I dati sono nella tabella `studente`.
- ▶ Bisogna filtrare le righe in base alla classe.
- ▶ Serve la clausola `WHERE`.

Soluzione esercizio 2

Query

```
SELECT nome, cognome  
FROM studente  
WHERE classe = '5IA';
```

Spiegazione

- ▶ WHERE classe = '5IA' seleziona solo gli studenti della classe richiesta.
- ▶ Il valore '5IA' è testuale, quindi va scritto tra apici singoli.
- ▶ La query mostra solo le colonne nome e cognome.

Esercizio 3

Testo

Visualizzare nome, cognome e voto di tutti gli esami sostenuti dagli studenti.

Idea di soluzione

- ▶ Il nome dello studente è nella tabella `studente`.
- ▶ Il voto è nella tabella `esame`.
- ▶ Bisogna collegare le due tabelle con un **equi join**.

Soluzione esercizio 3

Query con equi join

```
SELECT s.nome, s.cognome, e.voto  
FROM studente AS s, esame AS e  
WHERE s.id = e.studenteId;
```

Spiegazione

- ▶ s è l'alias della tabella studente.
- ▶ e è l'alias della tabella esame.
- ▶ s.id = e.studenteId è la condizione di equi join.
- ▶ La chiave primaria di studente viene confrontata con la chiave esterna di esame.

Testo

Visualizzare nome e cognome degli studenti che hanno preso un voto almeno pari a 8.

Idea di soluzione

- ▶ Bisogna collegare studente ed esame.
- ▶ Il collegamento avviene con equi join.
- ▶ Oltre al collegamento, serve un filtro sul voto.

Query con equi join

```
SELECT s.nome, s.cognome, e.voto
FROM studente AS s, esame AS e
WHERE s.id = e.studenteId
      AND e.voto >= 8;
```

Spiegazione

- ▶ `s.id = e.studenteId` collega studenti ed esami.
- ▶ `AND e.voto >= 8` aggiunge un filtro sui voti.
- ▶ La clausola `WHERE` contiene sia il collegamento sia la condizione richiesta.

Esercizio 5

Testo

Visualizzare nome dello studente, cognome, nome del corso e voto ottenuto.

Idea di soluzione

- ▶ Servono dati da tre tabelle: studente, esame, corso.
- ▶ esame collega studenti e corsi.
- ▶ Servono due condizioni di equi join.

Soluzione esercizio 5

Query con equi join

```
SELECT s.nome, s.cognome, c.nome AS corso, e.voto
FROM studente AS s, esame AS e, corso AS c
WHERE s.id = e.studenteId
      AND c.id = e.corsoId;
```

Spiegazione

- ▶ `s.id = e.studenteId` collega ogni esame al relativo studente.
- ▶ `c.id = e.corsoId` collega ogni esame al relativo corso.
- ▶ `AS corso` rinomina la colonna `c.nome`.

Testo

Visualizzare gli studenti della classe 5IB con il corso sostenuto e il voto ottenuto.

Idea di soluzione

- ▶ Servono tre tabelle.
- ▶ Bisogna collegarle con equi join.
- ▶ Bisogna aggiungere un filtro sulla classe.

Soluzione esercizio 6

Query con equi join

```
SELECT s.nome, s.cognome, c.nome AS corso, e.voto
FROM studente AS s, esame AS e, corso AS c
WHERE s.id = e.studenteId
      AND c.id = e.corsoId
      AND s.classe = '5IB';
```

Spiegazione

- ▶ Le prime due condizioni realizzano l'equi join.
- ▶ `AND s.classe = '5IB'` seleziona solo gli studenti della classe richiesta.
- ▶ Il risultato contiene solo esami sostenuti da studenti della 5IB.

Testo

Calcolare quanti esami sono stati registrati per ogni studente.

Idea di soluzione

- ▶ Bisogna collegare studente ed esame.
- ▶ Serve la funzione COUNT.
- ▶ Serve un raggruppamento per studente.
- ▶ Il collegamento deve essere scritto come equi join.

Query con equi join

```
SELECT s.nome, s.cognome, COUNT(e.id) AS numero_esami  
FROM studente AS s, esame AS e  
WHERE s.id = e.studenteId  
GROUP BY s.id, s.nome, s.cognome;
```

Spiegazione

- ▶ `s.id = e.studenteId` collega ogni esame allo studente.
- ▶ `COUNT(e.id)` conta gli esami di ciascuno studente.
- ▶ `GROUP BY` crea un gruppo per ogni studente.

Esercizio 8

Testo

Calcolare la media dei voti di ogni studente.

Idea di soluzione

- ▶ Bisogna collegare studente ed esame.
- ▶ La media si calcola con AVG.
- ▶ Serve un raggruppamento per studente.

Soluzione esercizio 8

Query con equi join

```
SELECT s.nome, s.cognome, AVG(e.voto) AS media_voti  
FROM studente AS s, esame AS e  
WHERE s.id = e.studenteId  
GROUP BY s.id, s.nome, s.cognome;
```

Spiegazione

- ▶ `s.id = e.studenteId` realizza l'equi join.
- ▶ `AVG(e.voto)` calcola la media dei voti.
- ▶ `GROUP BY` è necessario perché vogliamo una media separata per ogni studente.

Testo

Visualizzare per ogni corso il numero di esami registrati, mostrando solo i corsi con almeno due esami.

Idea di soluzione

- ▶ Bisogna collegare corso ed esame.
- ▶ Serve COUNT per contare gli esami.
- ▶ Serve GROUP BY per raggruppare per corso.
- ▶ Serve HAVING per filtrare i gruppi.

Soluzione esercizio 9

Query con equi join

```
SELECT c.nome AS corso, COUNT(e.id) AS numero_esami  
FROM corso AS c, esame AS e  
WHERE c.id = e.corsoId  
GROUP BY c.id, c.nome  
HAVING COUNT(e.id) >= 2;
```

Spiegazione

- ▶ `c.id = e.corsoId` collega ogni esame al corso corrispondente.
- ▶ `COUNT(e.id)` conta gli esami associati a ciascun corso.
- ▶ `HAVING` filtra dopo il raggruppamento.
- ▶ `WHERE` non può filtrare direttamente il risultato di `COUNT`.

Testo

Per ogni classe e per ogni corso, calcolare la media dei voti, mostrando solo i gruppi con media almeno pari a 8.

Idea di soluzione

- ▶ Servono le tre tabelle: studente, esame, corso.
- ▶ Bisogna collegarle con equi join.
- ▶ La media si calcola con AVG.
- ▶ Bisogna raggruppare per classe e corso.
- ▶ Serve HAVING per filtrare le medie.

Soluzione esercizio 10

Query con equi join

```
SELECT s.classe, c.nome AS corso, AVG(e.voto) AS media_voti
FROM studente AS s, esame AS e, corso AS c
WHERE s.id = e.studenteId
      AND c.id = e.corsoId
GROUP BY s.classe, c.id, c.nome
HAVING AVG(e.voto) >= 8;
```

Spiegazione

- ▶ Le condizioni nel WHERE realizzano l'equi join tra le tre tabelle.
- ▶ GROUP BY s.classe, c.id, c.nome crea un gruppo per ogni coppia classe-corso.
- ▶ AVG(e.voto) calcola la media dei voti per ciascun gruppo.